GUIA VISUAL 065

GREP E A BUSCA DE PADRÕES

*Uma chave de API vazou no repositório e a pergunta era simples: em quantos lugares ela aparece? O `grep CHAVE *` devolveu `É um diretório` — o `*` é do shell e não desce um nível. O `grep -r CHAVE .` acertou o alvo e errou o alcance: dois minutos varrendo `.git`, `node_modules` e um venv, e uma parede com **Arquivo binário … coincide** e uma linha de mil e duzentos caracteres de bundle minificado. A informação estava lá, **afogada**.*

065
GUIAS LINUX

AS OPÇÕES EM TRÊS ESTÁGIOS — DIAGNOSTIQUE O RESULTADO RUIM PELO ESTÁGIO

1 · COMO INTERPRETAR

- `-E` ERE — o dialeto do dia a dia
- `-F` literal: **nada** é especial, e é o mais rápido
- `-P` PCRE (`\d` , `\K` , `.*?`)
- `-i` ignora caixa
- `-e` mais um padrão · e resolve padrão que começa com traço
- `-f` padrões vindos de arquivo

2 · QUAIS LINHAS PASSAM

- `-v` inverte: imprime as que **não** casam
- `-w` palavra inteira — `root` não casa `chroot`
- `-x` a linha **inteira** é exatamente isso
- `-m N` para depois de N linhas
- `-A -B -C` N linhas depois · antes · dos dois lados

3 · O QUE IMPRIMIR

- `-o` só o trecho casado — vira **extrator**
- `-c` só a contagem de **linhas**
- `-l` / `-L` só o nome do arquivo que tem / não tem
- `-n` número da linha — o que dá citação de relatório
- `-h` / `-H` nunca / sempre o nome do arquivo

FORA DOS TRÊS ESTÁGIOS: ONDE PROCURAR

`-r` percorre diretórios (`-R` ainda segue link simbólico, com risco de laço — prefira `-r`) · `--include='*.conf'` e `--exclude-dir={.git,node_modules,venv}` filtram **arquivos, não linhas**, e aceitam **glob**, nunca expressão regular · `-I` pula binário. Em árvore de código, **esta linha faz mais pela qualidade do resultado que qualquer refinamento de padrão**: `grep -rnI 'AKIA3XPLR7QF2' . --exclude-dir={.git,node_modules} --include='*.py' --include='*.env'` .

Quase todo mau resultado vem de **mexer no estágio errado**. "Veio ruído demais" costuma ser o estágio 2 (falta `-w`) ou o bloco vermelho (falta `--exclude-dir`); "veio linha longa demais" é o estágio 3 (falta `-o`); **"não veio nada" é o estágio 1** — padrão escrito em ERE entregue a um `grep` em BRE. E o modelo por trás de tudo: **o `grep` decide linha por linha**. Padrão não atravessa quebra de linha, a saída sai na ordem da entrada e **não vem deduplicada** — ordenar e contar é trabalho do `sort` e do `uniq` .

NA PRÁTICA

kali@kali: ~/lab-grep

```
# -c conta LINHAS; para contar OCORRÊNCIAS, é -o | wc -l
$ grep -c 'ip=' acesso.log
3
$ grep -o 'ip=[0-9.]*' acesso.log | wc -l
4
# o que está DE FATO em vigor na configuração
$ grep -vE '^\s*(#|$)' /etc/ssh/sshd_config
# -o + sort + uniq -c: o pipeline canônico de contagem
$ grep -oE 'ip=[0-9.]+' acesso.log | cut -d= -f2 \
    | sort | uniq -c | sort -rn
      2 10.0.0.1
      1 10.0.0.2
# o grep que casa consigo mesmo — e a classe que resolve
$ ps aux | grep '[n]ginx'
```

Pegadinhas que custam tempo. `grep -c` com **vários arquivos** imprime `arquivo:contagem` por arquivo, nunca um total. Com **um** arquivo o `grep` não prefixa o nome, com **dois ou mais** prefixa — o que quebra script cujo `--include` às vezes casa um, às vezes vários: force com **`-h`** ou **`-H`**. Blocos de `-A` / `-B` vêm separados por uma linha `--` , ótima para ler e péssima para processar. **Cor vaza para o *pipeline*:** um `--color=always` herdado de *alias* injeta escapes ANSI que quebram o `sort` e o `awk` seguintes — o padrão sensato é **`--color=auto`**. E `grep padrao` sem arquivo **não travou**: está lendo o `stdin` — `Ctrl+D` encerra.

- **0** **Achou** pelo menos uma linha.
- **1** **Não achou nada.** Não é erro — é resposta.
- **2** **Erro real:** arquivo inexistente, padrão inválido.

O status de saída é informação, não formalidade — é ele que permite usar o `grep` como teste: `if grep -q '^PermitRootLogin yes' /etc/ssh/sshd_config; then …` . O `-q` é silencioso **e encerra na primeira linha que casa**: num arquivo de gigabytes, muda o tempo de minutos para milissegundos, e é sempre preferível a `grep … > /dev/null` .

Desempenho, em ordem de impacto: `LC_ALL=C` desliga a decodificação multibyte e costuma render de duas a cinco vezes em texto ASCII (não use com acento) · use o **dialeto mais fraco que resolva** — `-F` > `-E` > `-P` · **filtre barato antes de caro**: um `-F` que descarta 90% das linhas antes de um `-P` complexo vale mais que o inverso · `-q` e `-m 1` param cedo quando a pergunta é só "existe?". E `-F -f lista.txt` casa **milhares** de padrões literais numa **única** varredura: um laço com mil `grep` faz mil varreduras do mesmo log.

Duas combinações que derrubam script ou terminal. 1) `grep -q` sob `set -e` : "não encontrou" é status **1**, e o `set -e` lê isso como falha e **encerra o script** numa linha que só queria perguntar. Use dentro de `if` , ou acrescente `|| true` . **2)** `grep -a` trata binário como texto e pode **despejar bytes de controle** no terminal, deixando-o ilegível — se acontecer, `reset` conserta; a defesa é `-I` , que pula binário por completo. Para casar padrão que atravessa linhas, `-z` usa o byte nulo como separador de registro: `grep -zoP 'BEGIN CERTIFICATE.*?END CERTIFICATE' arquivo.pem | tr -d '\0'` .

RESUMO RÁPIDO

- ✓ **O `grep` decide por linha e nada mais.** Padrão não atravessa quebra de linha (salvo `-z`); a saída sai na ordem de entrada e não vem deduplicada.
- ✓ **Status: 0 achou · 1 não achou · 2 erro.** Use `grep -q` em teste — mas o status 1 encerra script sob `set -e`.
- ✓ **As opções se dividem em estágios:** interpretar (`-E -F -P -i`), quais linhas passam (`-v -w -x -m -A -B -C`), o que imprimir (`-o -c -l -n -h`) e onde procurar (`-r --include --exclude-dir -I`).
- ✓ **`-c` conta linhas, não ocorrências.** Para ocorrências, `grep -o | wc -l`.
- ✓ **`-o` transforma o `grep` em extrator** — é a peça que o encaixa no pipeline com `sort`, `uniq -c` e `awk`.
- ✓ **`-w` e `-x` matam os dois falsos positivos mais comuns** sem escrever padrão nenhum.
- ✓ **Em árvore de código, `--exclude-dir={.git,node_modules}` e `-I` valem mais que refinar o padrão.**
- ✓ **Desempenho:** `LC_ALL=C`, dialeto mais fraco possível, barato antes de caro, `-q` / `-m 1` para "existe?".

NO KALI

Varrer um alvo baixado é o uso mais frequente. Depois de espelhar um site com `wget -r`, um único `grep -rniI --include='*.js' -E 'api[_-]?key|secret|token|password' ./espelho/` levanta credencial exposta em segundos: o `-I` evita afogar a saída em imagem, o `-i` cobre variação de caixa e o `--include` restringe ao que interessa. Vale para material de **teste autorizado ou de escopo público** — vasculhar sistema de terceiro sem contrato é acesso não autorizado pela **Lei 12.737/2012**. **`grep -F -f iocs.txt acesso.log` é o casamento em massa:** com uma lista de IPs ou domínios maliciosos num arquivo, os dois conjuntos se comparam numa varredura só; mil chamadas num laço fazem mil varreduras — em log de gigabytes, a diferença é entre segundos e uma tarde. **Refinar wordlist antes da ferramenta economiza horas:** `grep -E '^.{8,16}$' rockyou.txt | grep -vE '^[0-9]+$' > filtrada.txt` reduz milhões de candidatos ao que a política de senha do alvo permite — menos requisições, menos ruído no log do alvo e menos risco de derrubar um serviço que o contrato manda preservar. **Auditar serviço são dois comandos:** `grep -vE '^\s*(#|$)' /etc/ssh/sshd_config` mostra o que está em vigor, e `grep -niE '^\s*(PermitRootLogin|PasswordAuthentication|X11Forwarding)'` vai direto às diretivas — o `-n` é o que transforma o achado em **referência citável no relatório**. **No shell de um alvo comprometido, `grep` é o que existe:** não conte com `rg`, `jq` ou Python num contêiner mínimo, e saiba que o `grep` do BusyBox **não tem `-P` nem `--exclude-dir`** — o plano B é `find … -print0 | xargs -0 grep`. No Kali há o `rg` (multiencadeado, respeita `.gitignore`, pula binário sem opção nenhuma); é ótimo em casa e ausente no alvo.